React Fundamentals — Q&A

A quick-reference of core React concepts, with examples drawn from the Expense Tracker app.

1. Why can't you mutate state directly?

React only re-renders when it detects a state change, and its detection is based on **reference comparison** (`Object.is(oldValue, newValue)`), not deep equality.

```
// ❌ Mutation — same reference, no re-render
expenses.push(newExpense);
setExpenses(expenses);

// ✅ New reference — React sees the change, re-renders
setExpenses([...expenses, newExpense]);
```

- `.push` , `.splice` , `.sort` , `obj.prop = ...` all modify the existing object/array. The reference is unchanged, so React's bail-out check (`Object.is(old, new) === true`) skips the render.
- `[...arr, item]` , `arr.filter(...)` , `arr.map(...)` , `{ ...obj, key: value }` all return **new** references. React sees `old !== new` and re-renders.

Beyond the re-render issue, mutation also breaks:

- **Time-travel debugging** in React DevTools (previous snapshots become corrupted).
- `React.memo` / `useMemo` / `useCallback` (they all rely on reference identity).
- **Concurrent rendering** (React may compare mid-render values against prior state).

Rule: treat state as immutable. Always produce a new object/array when updating.

2. What does the `key` prop do, and why not use the index?

`key` is React's identity tag for elements in a list. It tells React "this specific element in the previous render is the same as this specific element in the new render."

```
{expenses.map((expense) => (
    <li key={expense.id}>{expense.name}</li>
))}
```

React uses keys to:

- **Reuse DOM nodes** instead of destroying and recreating them.
- **Preserve component state** across re-renders (form input focus, animation state, `useState` inside a list item).

- **Efficiently reorder** items when the array changes.

Why not the array index?

Using `key={index}` breaks the identity model whenever items are added, removed, or reordered:

```
// Items: [A, B, C], keys: [0, 1, 2]
// User deletes A → [B, C], keys: [0, 1]
// React thinks: "key 0 changed from A to B" — treats it as an update, not a deletion
```

Consequences:

- Wrong item's state (input value, focus, checkbox) shows up.
- DOM nodes get re-used against your intent → subtle animation/focus bugs.
- Deletion looks correct visually but internally it's actually mutating existing elements.

Index is fine **only if** the list is truly static — never sorted, filtered, added to, or removed from. For anything dynamic, use a stable unique id like `crypto.randomUUID()` (as this project does).

3. Difference between props and state?

	Props	State
Who owns it?	The parent	The component itself
Mutable?	Read-only from the child's view	Mutable via `useState` setter
Trigger re-render?	Parent re-renders when it changes	Own component re-renders when setter is called
Analogy	Function arguments	Local variables that persist across calls

Example from this app:

```
// App.tsx (parent)
const [expenses, setExpenses] = useState<Expense[]>([]);   // state (owned by App)
<ExpenseList expenses={expenses} onDeleteExpense={deleteExpense} />
//            ^^^^^^^^ prop (received by ExpenseList)
```

`ExpenseList` cannot call `setExpenses` — that would break encapsulation. It can only receive `expenses` as a prop and *ask* `App` to change things via the `onDeleteExpense` callback (also a prop).

Rule of thumb: state lives at the lowest common ancestor of everything that needs to read or update it. Everything else receives it as a prop.

4. `setCount(count + 1)` vs `setCount(prev => prev + 1)` — when does it matter?

Direct form — reads `count` from the current closure

```
setCount(count + 1);
```

Functional form — reads `count` from React's latest queued state

```
setCount(prev => prev + 1);
```

They behave identically for a single call. The difference shows up when **multiple updates are queued in the same event, or an update runs after a delay**:

```
// Direct form — all three reads see the same stale `count`
setCount(count + 1);
setCount(count + 1);
setCount(count + 1);
// count was 0, ends at 1 (not 3!)

// Functional form — each call sees the result of the previous
setCount(prev => prev + 1);
setCount(prev => prev + 1);
setCount(prev => prev + 1);
// count was 0, ends at 3
```

Why? Inside an event handler, all `setState` calls are **batched**. The direct form captures `count` from the closure at handler-start time. The functional form receives the up-to-date value from React each time.

The classic bug:

```
setTimeout(() => setCount(count + 1), 1000);
// `count` is captured from when the timer was set,
// not from the moment the timer fires — could be badly stale
```

Rule: if the new state depends on the old state, use the functional form. That's why every setter in this app (`setExpenses` , `setErrors`) uses `prev =>` .

5. Controlled vs uncontrolled inputs — pros and cons?

Controlled — React owns the value

```
<input value={name} onChange={(e) => setName(e.target.value)} />
```

React's state drives what the input shows. The DOM and state can never disagree — the input is a *rendering* of state.

Uncontrolled — DOM owns the value

```
const inputRef = useRef<HTMLInputElement>(null);
// ...later
console.log(inputRef.current?.value);
```

The DOM tracks the value internally. React reads it via a ref only when needed (usually on submit).

Comparison

	Controlled	Uncontrolled
Live validation	Easy — validate in `onChange`	Awkward — need `onBlur` or read the ref
Reset / prefill	`setName("")`	`inputRef.current.value = ""` (imperative)
Instant UI updates	Trivial (button `disabled` reacts)	Requires extra effort
Performance	Re-renders every keystroke	Zero React overhead
Integration with libraries	Standard	Sometimes required (file inputs, some 3rd-party)
Complexity	Slightly more code	Less code, but less flexibility

Rule of thumb:

- Small-to-medium forms with live validation, dependent fields, or dynamic UI → **controlled** (like this app's `ExpenseForm`).
- Very large forms with performance issues, or one-off values that are only read on submit → **uncontrolled**, or use a library like React Hook Form (which uses uncontrolled inputs under the hood for speed).

`<input type="file">` is *always* uncontrolled because the browser doesn't let JavaScript set its value programmatically.

6. Why does `e.preventDefault()` matter in a React form?

The default browser behavior for a `<form>` submit is:

1. Serialize form fields into a query string or `FormData`.
2. Make a full-page HTTP request to the form's `action` URL.
3. **Reload the page**, discarding your React app's state and re-mounting everything.

That's fine for classic server-rendered pages. In a React SPA, it's catastrophic — you lose your `expenses` array, your form state, everything.

```
function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();   // ← stops the reload
    onAddExpense(newExpense);
}
```

`preventDefault()` cancels the browser's built-in behavior so *only* your JS runs. The URL doesn't change; nothing reloads; React state survives.

Alternatives (and why the form pattern is still preferred)

You could put an `onClick` on the button instead of `onSubmit` on the form. But then:

- Pressing **Enter** in a text field won't submit — you lose keyboard accessibility.
- Screen readers won't announce it as a proper form.

Sticking with `<form onSubmit={...}>` + `e.preventDefault()` gives you accessibility and native keyboard support for free.

7. How do you share state between two sibling components?

You can't — directly. Siblings have no reference to each other. You **lift the state up** to their nearest common ancestor.

```
         App                    App owns `expenses`
        /   \
ExpenseForm  ExpenseList        Both siblings read/write via props + callbacks
```

- `ExpenseForm` reports "user added an expense" up via `onAddExpense` prop.
- `App` updates `expenses` state.
- `App` passes the new `expenses` down to `ExpenseList` as a prop.
- `ExpenseList` re-renders with the new data.

This is *the* central pattern in React. Every "how do X and Y talk to each other" answer starts with "find their common ancestor."

When lifting becomes painful

If state needs to be shared between distant components (5+ levels of nesting), passing it through every intermediate component ("prop drilling") gets tedious. Solutions in order of complexity:

1. **Component composition** — pass children as props to avoid drilling one layer.
2. `useContext` — for genuinely global state (theme, auth user, locale).
3. **State libraries** (Zustand, Redux, Jotai) — for large apps with many independent state slices.

For this app, plain lifting to `App` is exactly right — the tree is small.

8. How does a child update its parent's state?

By calling a **callback prop** the parent passed down. The child never touches the parent's setter directly.

```
// App.tsx (parent)
function addExpense(expense: Expense) {
    setExpenses(prev => [...prev, expense]);
}
<ExpenseForm onAddExpense={addExpense} />

// ExpenseForm.tsx (child)
type Props = { onAddExpense: (expense: Expense) => void };
function ExpenseForm({ onAddExpense }: Props) {
    function handleSubmit() {
        onAddExpense(newExpense);   // ← child asks parent to update
    }
}
```

The pattern:

- **Data flows down** (parent → child) via props.
- **Events flow up** (child → parent) via callback props.

This is called **unidirectional data flow** and it's why React apps are easier to reason about than two-way-binding frameworks. You always know where a piece of state lives and who can change it.

Why not pass `setExpenses` itself?

You could:

```
<ExpenseForm setExpenses={setExpenses} />
```

But it's bad practice because:

1. **Too much power** — the child could replace, clear, or corrupt the array. The contract is too broad.
2. **Tight coupling** — the child now knows the parent uses `useState` with an array. If the parent later switches to `useReducer` or Redux, the child breaks.

3. **Hard to test** — you have to mock a setter instead of a plain function.

A narrow callback like `onAddExpense(expense)` limits the child's power to exactly what it needs.

9. When should you store something in state vs. compute during render?

Compute during render whenever possible. Storing derived values in state creates two sources of truth that can drift out of sync.

Ask yourself:

> Can I calculate this from existing props or state?

If yes → compute it in the render body. If no → store it in state.

Examples from this app

```
// ✅ Derived — recomputed each render
const visibleExpenses = catFilter === "All"
    ? expenses
    : expenses.filter(e => e.category === catFilter);

const total = expenses.reduce((sum, e) => sum + e.amount, 0);

const isFormValid = name.trim() !== "" && Number(amount) > 0;


// ✅ Actual state — user-entered input, can't be derived from anything else
const [name, setName] = useState("");
const [expenses, setExpenses] = useState<Expense[]>([]);
const [catFilter, setCatFilter] = useState<FilteringCategory>("All");
```

The anti-pattern

```
// ❌ Derived value stored in state
const [expenses, setExpenses] = useState([]);
const [total, setTotal] = useState(0);

function addExpense(e) {
    setExpenses(prev => [...prev, e]);
    setTotal(prev => prev + e.amount);   // must remember to update this
}
```

```
function deleteExpense(id) {
    setExpenses(prev => prev.filter(e => e.id !== id));
    // ← forgot to update total! Now they're out of sync.
}
```

Two states → two opportunities to forget → drift. Compute `total` from `expenses` and it's guaranteed correct forever.

When storing derived data *is* okay

- **Expensive to compute** and it becomes visibly slow — wrap in `useMemo` (cache the derived value between renders, not really "state").
- **Snapshot semantics** — you *want* to freeze the derived value at a moment in time (e.g. "checkout total" that shouldn't recompute if the cart changes mid-purchase).

Otherwise: derive.

10. What triggers a re-render in React?

A component re-renders when one of these happens:

1. **Its own state changes** (a `useState` setter is called with a *different* value, per `Object.is`).
2. **Its parent re-renders** (which passes it possibly-new props — React re-runs the child unconditionally unless you wrap it in `React.memo`).
3. **A context it consumes changes** (a `useContext` value provider's value updates).
4. **A `useSyncExternalStore` subscription fires** (used by state libraries like Zustand).

What does *not* trigger a re-render

- **Mutating an object or array in state** — same reference → React skips the render (see Q1).
- **Setting state to the same value** — `setCount(5)` when count is already `5` → bail-out.
- **Updating a ref** — `ref.current = ...` is intentionally invisible to the render cycle. Use refs for values you want to persist without causing renders (DOM handles, timers, previous values).
- **Changes to plain variables outside components** — module-level `let`s don't trigger anything.

Cascading renders

When `App` re-renders (say, because `expenses` changed):

- Every direct child (`ExpenseForm`, `ExpenseList`, `TotalExpense`) also re-runs, even if their props didn't change.
- React then reconciles the resulting virtual DOM. If a child's props were actually the same and it's wrapped in `React.memo`, React skips re-running it. Otherwise it just re-runs the function — but if the resulting JSX is unchanged, the DOM isn't touched.

Key insight: "re-render" means "React re-runs the component function and diffs the result against the previous virtual DOM." It does *not* automatically mean "the DOM updates." Cheap function calls + smart

diffing is what makes React fast.

Cheat-sheet

Concept	Rule
Mutation	Never. Always return new references.
Keys	Stable, unique, from data. Never index for dynamic lists.
Props vs state	Props: from parent, read-only. State: owned, mutable via setter.
Setter with prior state	Use functional form `prev => ...` .
Inputs	Prefer controlled. Uncontrolled for perf or file inputs.
Form submit	Always `e.preventDefault()` .
Sibling communication	Lift state to common ancestor.
Child → parent update	Callback prop, not the setter itself.
Derived data	Compute in render, don't store in state.
Re-render triggers	Own state, parent render, context change, external store update.